1. Delete a node from a Binary Search Tree

```
class BST {
  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }
  Node root;
  Node insert(Node root, int data) {
    if (root == null)
      return new Node(data);
    if (data < root.data)
      root.left = insert(root.left, data);
    else
      root.right = insert(root.right, data);
    return root;
  }
  Node delete(Node root, int key) {
    if (root == null)
      return null;
    if (key < root.data) {
      root.left = delete(root.left, key);
    }
    else if (key > root.data) {
      root.right = delete(root.right, key);
    }
    else {
      if (root.left == null)
        return root.right;
      if (root.right == null)
        return root.left;
      Node temp = root.right;
      while (temp.left != null)
        temp = temp.left;
      root.data = temp.data;
      root.right = delete(root.right, temp.data);
    }
    return root;
  }
  void inorder(Node root) {
    if (root == null)
      return;
     inorder(root.left);
```

```
    System.out.print(root.data + " ");
    inorder(root.right);
  }
  public static void main(String[] args) {
    BST tree = new BST();
    tree.root = tree.insert(tree.root, 50);
    tree.insert(tree.root, 30);
    tree.insert(tree.root, 70);
    tree.insert(tree.root, 20);
    tree.insert(tree.root, 40);
    tree.insert(tree.root, 60);
    tree.insert(tree.root, 80);
    System.out.print("Before deletion: ");
    tree.inorder(tree.root);
    tree.root = tree.delete(tree.root, 50);
    System.out.print("\nAfter deletion: ");
    tree.inorder(tree.root);
  }
}
```

2. Check if a Binary Tree is a Binary Search Tree

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static boolean isBST(Node root, long min, long max) {

    if (root == null)
      return true;

    if (root.data <= min || root.data >= max)
      return false;

    return isBST(root.left, min, root.data)
      && isBST(root.right, root.data, max);
  }
```

```
  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    Node root = new Node(50);

    root.left = new Node(30);
    root.right = new Node(70);

    root.left.left = new Node(20);
    root.left.right = new Node(40);

    root.right.left = new Node(60);
    root.right.right = new Node(80);

    if (isBST(root, Long.MIN_VALUE, Long.MAX_VALUE))
      System.out.println("BST");
    else
      System.out.println("Not BST");
  }
}
```

3. Find Kth Smallest and Kth Largest Element in a BST

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static Node insert(Node root, int data) {
    if (root == null)
      return new Node(data);

    if (data < root.data)
      root.left = insert(root.left, data);
    else
      root.right = insert(root.right, data);

    return root;
```

```
  }

  static void inorder(Node root, ArrayList<Integer> list) {
    if (root == null)
      return;

    inorder(root.left, list);
    list.add(root.data);
    inorder(root.right, list);
  }

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    Node root = null;

    int n = sc.nextInt();

    for (int i = 0; i < n; i++)
      root = insert(root, sc.nextInt());

    int k = sc.nextInt();

    ArrayList<Integer> list = new ArrayList<>();

    inorder(root, list);

    System.out.println("Kth Smallest: " + list.get(k - 1));

    System.out.println("Kth Largest: "
        + list.get(list.size() - k));
  }
}
```

4. Check Whether a BST is Balanced or Not

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
```

```
    }
}

static Node insert(Node root, int data) {
    if (root == null)
        return new Node(data);

    if (data < root.data)
        root.left = insert(root.left, data);
    else
        root.right = insert(root.right, data);

    return root;
}

static int height(Node root) {
    if (root == null)
        return 0;

    return Math.max(height(root.left),
                    height(root.right)) + 1;
}

static boolean isBalanced(Node root) {
    if (root == null)
        return true;

    int left = height(root.left);
    int right = height(root.right);

    if (Math.abs(left - right) > 1)
        return false;

    return isBalanced(root.left)
        && isBalanced(root.right);
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    Node root = null;

    int n = sc.nextInt();

    for (int i = 0; i < n; i++)
        root = insert(root, sc.nextInt());
```

```
    if (isBalanced(root))
      System.out.println("Balanced");
    else
      System.out.println("Not Balanced");
  }
}
```

SQL-TASK:

1.

```
SELECT emp_name, department, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

2.

```
SELECT emp_name, department, salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

3.

```
SELECT emp_name, department, salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

4.

```
SELECT emp_name, department, salary,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

5.

```
SELECT emp_name, department, salary,
    RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

6.

```
WITH ranked AS (
  SELECT emp_name, department, salary,
      DENSE_RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
      ) AS salary_rank
  FROM employees
)
```

```
SELECT emp_name, department, salary
FROM ranked
WHERE salary_rank <= 2;
```

7.

```
SELECT emp_name, department, salary,
    FIRST_VALUE(emp_name) OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS highest_paid_employee
FROM employees;
```

8.

```
SELECT emp_name, department, salary,
    LAST_VALUE(emp_name) OVER (
      PARTITION BY department
      ORDER BY salary DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS lowest_paid_employee
FROM employees;
```

9.

```
SELECT emp_name, department, salary,
    FIRST_VALUE(salary) OVER (
      PARTITION BY department
      ORDER BY salary DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS highest_salary,
    LAST_VALUE(salary) OVER (
      PARTITION BY department
      ORDER BY salary DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS lowest_salary
FROM employees;
```

10.

```
WITH ranked AS (
  SELECT emp_name, department, salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
  FROM employees
)
SELECT emp_name, department, salary
FROM ranked
WHERE salary_rank = 2;
```